containerized application, that component will configure the JVM memory based on the resource limits assigned to the container. If you don't specify any limits, the results will be unpredictable, which is not what you want.

There's also an economic aspect to consider. If you run your applications in a public cloud, you're usually charged based on how many resources you consume. Consequently, you'll probably want to be in control of how much CPU and memory each of your containers can use to avoid nasty surprises when the bill arrives.

When it comes to orchestrators like Kubernetes, there's another critical issue related to resources that you should consider. Kubernetes schedules Pods to be deployed in any of the cluster nodes. But what if a Pod is assigned to a node that has insufficient resources to run the container correctly? The solution is to declare the minimum CPU and memory a container needs to operate (*resource requests*). Kubernetes will use that information to deploy a Pod to a specific node only if it can guarantee the container will get at least the requested resources.

Resource requests and limits are defined per container. You can specify both requests and limits in a Deployment manifest. We haven't defined any limits in the base manifests for Catalog Service because we've been operating in a local environment and we didn't want to constrain it too much in terms of resource requirements. However, production workloads should always contain resource configurations. Let's look at how we can do that for the production deployment of Catalog Service.

ASSIGNING RESOURCE REQUESTS AND LIMITS TO A CONTAINER